

LABORATORIO PRÁCTICO

Gestión de Grandes Volúmenes de Datos con Google Cloud Platform

BigQuery · Cloud Storage · Cloud Data Fusion · Dataflow



Duración
90 minutos



Nivel
Intermedio — Big Data

Caso de Uso: Flota de Buses — Santiago, Chile

Análisis de posicionamiento GPS para responder preguntas de negocio

Descripción General del Laboratorio

En este laboratorio trabajarás con datos reales de posicionamiento GPS de una flota de buses. Aprenderás a ingestar datos en bruto a un Data Lake en Cloud Storage, crear un Data Warehouse en BigQuery con estructura profesional, ejecutar un pipeline de transformación y limpieza con Cloud Data Fusion o Dataflow, y responder preguntas de negocio mediante consultas SQL.

Arquitectura de la Solución

FLUJO DE DATOS			
1. RAW LAYER Archivos Parquet en Cloud Storage (GCS)	2. STAGING Pipeline ETL en Data Fusion / Dataflow	3. GOLD LAYER Tabla limpia en BigQuery DW	4. REPORTING Vistas SQL para Preguntas de Negocio

Tiempos Sugeridos

Paso 1 — Cloud Storage	15 min — Crear bucket y subir archivos Parquet manualmente
Paso 2 — BigQuery DW	20 min — Crear datasets, tabla principal y tablas de control
Paso 3 — Pipeline ETL	35 min — Configurar Data Fusion o Dataflow, ejecutar carga
Paso 4 — Análisis SQL	20 min — Crear vistas y responder preguntas de negocio

Importante — Entorno de Google Skills

La sesión dura exactamente 90 minutos. Los recursos se eliminan automáticamente al finalizar.
Guarda tus consultas SQL en un bloc de notas local durante el laboratorio.
No cierres el navegador ni la sesión de Cloud Shell.

Paso 1 — Data Lake en Cloud Storage

Crearás un bucket en Google Cloud Storage que actuará como tu capa Raw del Data Lake. Los archivos Parquet se cargarán manualmente desde Cloud Shell.

1.1 Preparar el Entorno

Abre Cloud Shell en la consola de GCP y ejecuta el siguiente bloque para verificar las restricciones de tu proyecto y definir las variables de entorno:

```
# Verificar restricciones de localización del proyecto
gcloud org-policies describe constraints/gcp.resourceLocations \
  --project=$(gcloud config get-value project)

# Confirmar con 'yes' si se solicita
```

Nota sobre la región

Observa qué regiones están permitidas en tu proyecto.

Este laboratorio usa us-central1, que está disponible en la mayoría de los entornos de Google Skills.

1.2 Crear el Bucket GCS

Define las variables y crea el bucket. El nombre incluirá tu PROJECT_ID para garantizar unicidad global:

```
export PROJECT_ID=$(gcloud config get-value project)
export BUCKET_NAME="buses-datalake-${PROJECT_ID}"
export REGION="us-central1"

echo "Proyecto : $PROJECT_ID"
echo "Bucket   : gs://$BUCKET_NAME"
echo "Región    : $REGION"

# Crear el bucket
gcloud storage buckets create gs://$BUCKET_NAME \
  --location=$REGION \
  --project=$PROJECT_ID

# Verificar que el bucket fue creado
gcloud storage buckets describe gs://$BUCKET_NAME
```

1.3 Descargar y Subir los Archivos de Datos

Descarga los archivos del repositorio del curso y súbelos al bucket con la estructura de partición correcta:

```
# Clonar el repositorio con los datos
git clone https://github.com/jorgeanais/bigdata_demo_ev2
cd bigdata_demo_ev2/

# Descargar los archivos Parquet
python resources/download_data.py
cd data/
```

```
# Verificar que los archivos existen
ls -lh *.parquet
```

```
# Subir los tres archivos al Data Lake con estructura de partición por fecha
gcloud storage cp buses_2026-05-08.parquet \
  gs://$BUCKET_NAME/raw/buses/date=2026-05-08/

gcloud storage cp buses_2026-05-09.parquet \
  gs://$BUCKET_NAME/raw/buses/date=2026-05-09/

gcloud storage cp buses_2026-05-10.parquet \
  gs://$BUCKET_NAME/raw/buses/date=2026-05-10/

# Verificar estructura del Data Lake
gcloud storage ls --recursive gs://$BUCKET_NAME/raw/
```

✓ Verificación del Paso 1

Debes ver tres archivos listados, uno por cada fecha.

La estructura debe ser: gs://[BUCKET]/raw/buses/date=YYYY-MM-DD/[archivo].parquet

Esta convención de nombre de carpeta es compatible con particionamiento Hive.

Paso 2 — Data Warehouse en BigQuery

Crearás la estructura del Data Warehouse en BigQuery. Utilizarás dos datasets: buses_dw para los datos procesados y buses_control para la trazabilidad de los pipelines.

2.1 Crear los Datasets

Ejecuta en Cloud Shell:

```
# Dataset principal – capa Gold
bq mk --dataset --location=US \
  --description="Data Warehouse flota de buses" \
  ${PROJECT_ID}:buses_dw

# Dataset de control y trazabilidad
bq mk --dataset --location=US \
  --description="Tablas de control y registro de ejecución" \
  ${PROJECT_ID}:buses_control

# Verificar que ambos datasets fueron creados
bq ls --project_id=${PROJECT_ID}
```

2.2 Crear la Tabla Principal — Capa Gold

Ejecuta esta DDL en BigQuery > Editor de Consultas. Esta tabla es el destino final del pipeline ETL:

```
CREATE TABLE IF NOT EXISTS buses_dw.bus_positions (

  -- Identificación del bus
  patente          STRING,
  servicio          STRING,
  operador          INT64,
  direccion         STRING,

  -- Posición GPS
  latitud           FLOAT64,
  longitud           FLOAT64,
  velocidad          FLOAT64,

  -- Timestamps
  timestamp_gps      TIMESTAMP,
  timestamp_captura  TIMESTAMP,
  fecha_particion    DATE,      -- columna derivada para partición

  -- Columnas calculadas por el pipeline
  hora_captura       INT64,      -- 0-23
  dia_semana         STRING,     -- Lunes, Martes, ...
  es_hora_punta      BOOL,       -- 7-9h o 17-19h
  latencia_segundos  FLOAT64,    -- diferencia GPS vs captura

  -- Flags de calidad de datos
  velocidad_valida   BOOL,       -- velocidad entre 0 y 120 km/h
  gps_valido         BOOL,       -- coordenadas dentro de Chile
```

```
-- Trazabilidad
archivo_origen      STRING,      -- nombre del archivo Parquet fuente
procesado_en        TIMESTAMP    -- cuándo fue procesado
)
PARTITION BY fecha_particion
CLUSTER BY servicio, operador, patente;
```

Importante — no usar NOT NULL en ninguna columna

Apache Beam WriteToBigQuery declara todas las columnas como NULLABLE en el schema del job. Si la tabla tiene columnas REQUIRED (NOT NULL), BigQuery rechaza el load con error de schema mismatch.

Todas las columnas de esta tabla deben ser NULLABLE para que el pipeline funcione correctamente.

2.3 Tabla de Control de Pipeline

Registra cada ejecución del pipeline con su estado y estadísticas:

```
CREATE TABLE IF NOT EXISTS buses_control.pipeline_log (
  archivo_nombre      STRING      NOT NULL,
  fecha_datos         DATE        NOT NULL,
  estado              STRING      NOT NULL,  -- 'SUCCESS', 'FAILED', 'SKIPPED'
  filas_procesadas     INT64,
  filas_rechazadas     INT64,
  inicio_proceso      TIMESTAMP,
  fin_proceso         TIMESTAMP,
  mensaje_error       STRING
);
```

2.4 Tabla de Registros Inválidos

Almacena los registros que no superaron la validación, manteniendo trazabilidad completa:

```
CREATE TABLE IF NOT EXISTS buses_control.registros_invalidos (
  patente            STRING,
  latitud            FLOAT64,
  longitud           FLOAT64,
  velocidad          FLOAT64,
  servicio           STRING,
  timestamp_gps      STRING,
  archivo_origen     STRING,
  motivo_rechazo     STRING,
  procesado_en       TIMESTAMP
);
```

💡 Diseño del DW — Decisiones clave

PARTITION BY fecha_particion: BigQuery solo leerá las particiones necesarias, reduciendo costo y tiempo.

CLUSTER BY servicio, operador, patente: Ordena físicamente los datos para consultas frecuentes.

buses_control: Permite idempotencia — el pipeline puede verificar si ya procesó un archivo.

Paso 3 — Pipeline ETL con Python y Apache Beam

Construirás el pipeline ETL usando Python con Apache Beam. El script lee los archivos Parquet desde GCS, aplica transformaciones y validaciones, escribe los registros válidos en BigQuery y los inválidos en una tabla de control. El pipeline es idempotente: verifica si un archivo ya fue procesado antes de ejecutar.

Importante — Qwiklabs bloquea cambios IAM en Dataflow

Los proyectos de Qwiklabs tienen org policies que impiden asignar roles IAM manualmente.

Intentar configurar permisos de Dataflow resultará en SIN BINDINGS sin importar cuantas veces se intente.

En este laboratorio usaremos DirectRunner, que ejecuta el pipeline localmente en Cloud Shell.

DirectRunner no requiere permisos IAM adicionales y funciona de inmediato.

Tecnología	Python 3.11 + Apache Beam SDK con DirectRunner
Ejecución	El pipeline corre localmente en Cloud Shell (no requiere Dataflow)
Idempotencia	El script consulta pipeline_log antes de procesar cada archivo
Trazabilidad	Cada registro cargado lleva archivo_origen y procesado_en

3.1 Preparar el Entorno en Cloud Shell

Habilita las APIs necesarias, instala las dependencias Python y prepara el directorio de trabajo:

```
# Habilitar APIs
gcloud services enable dataflow.googleapis.com
gcloud services enable bigquery.googleapis.com

# Instalar dependencias
pip install apache-beam[gcp] pyarrow pandas pytz --quiet

# Verificar
python -c "import apache_beam, pyarrow, pytz, google.cloud.bigquery; print('OK – dependencias listas')"
```

```
# Crear directorio de trabajo
mkdir -p ~/buses_etl && cd ~/buses_etl
```

3.2 Script del Pipeline — pipeline_buses.py

Crea el archivo con el siguiente contenido completo. Puedes usar nano o el editor de Cloud Shell:

```
# Crear el archivo del pipeline
cat > ~/buses_etl/pipeline_buses.py << 'PYEOF'
```

```
"""
Pipeline ETL – Flota de Buses
Lee archivos Parquet desde GCS, transforma y carga en BigQuery.
Implementa idempotencia mediante tabla de control pipeline_log.
"""

import apache_beam as beam
```

```

from apache_beam.options.pipeline_options import PipelineOptions, GoogleCloudOptions
from apache_beam.io.gcp.bigquery import WriteToBigQuery, BigQueryDisposition
from google.cloud import bigquery
from datetime import datetime, timezone
import pyarrow.parquet as pq
import pyarrow.fs as pafs
import argparse
import logging
import sys
import os

```

```

logging.basicConfig(level=logging.INFO)
log = logging.getLogger(__name__)

```

```

# -----
# Constantes de validación
# -----
VEL_MIN, VEL_MAX = 0.0, 120.0      # km/h válidos
LAT_MIN, LAT_MAX = -56.0, -17.0   # Bounding box Chile
LON_MIN, LON_MAX = -76.0, -65.0
DIAS_ES = {
    'Monday': 'Lunes', 'Tuesday': 'Martes', 'Wednesday': 'Miercoles',
    'Thursday': 'Jueves', 'Friday': 'Viernes',
    'Saturday': 'Sabado', 'Sunday': 'Domingo'
}

```

```

# -----
# Funciones de transformación (DoFn)
# -----

```

```

class TransformarRegistro(beam.DoFn):
    """Aplica transformaciones y calcula columnas derivadas."""

    OUTPUT_INVALIDO = 'invalido'

    def __init__(self, archivo_origen):
        self.archivo_origen = archivo_origen

    def process(self, row):
        """
        Recibe un dict con las columnas originales del Parquet.
        Emite al output principal si es válido,
        o al output 'invalido' si no pasa las validaciones.
        """
        from datetime import timezone
        import pytz

        ahora_utc = datetime.now(timezone.utc)
        motivos = []

        # --- Parsear timestamp_gps ---
        # Soporta formatos: '2026-05-08T20:41:49+0000', '2026-05-08 20:41:49', ISO
8601
        ts_gps = None
        try:
            raw = row.get('timestamp_gps', '')

```



```

        if raw is None or raw == '':
            motivos.append('timestamp_gps_nulo')
        elif isinstance(raw, str):
            raw_clean = raw.replace('+0000', '+00:00').replace('Z', '+00:00')
            try:
                ts_gps = datetime.fromisoformat(raw_clean)
                if ts_gps.tzinfo is None:
                    ts_gps = ts_gps.replace(tzinfo=datetime.timezone.utc)
            except ValueError:
                ts_gps = datetime.strptime(raw[:19], '%Y-%m-%dT%H:%M:%S')
                ts_gps = ts_gps.replace(tzinfo=datetime.timezone.utc)
            elif hasattr(raw, 'tzinfo'):
                ts_gps = raw if raw.tzinfo else raw.replace(tzinfo=datetime.timezone.utc)
        except Exception as e:
            motivos.append(f'timestamp_gps_invalido({e})')

    # --- timestamp_captura ---
    ts_cap = row.get('timestamp_captura')
    if ts_cap is None:
        motivos.append('timestamp_captura_nulo')

    # --- Validar velocidad ---
    vel = float(row.get('velocidad') or -1)
    vel_valida = VEL_MIN <= vel <= VEL_MAX
    if not vel_valida:
        motivos.append(f'velocidad_fuera_rango({vel})')

    # --- Validar coordenadas GPS ---
    lat = float(row.get('latitud') or 0)
    lon = float(row.get('longitud') or 0)
    gps_valido = (LAT_MIN <= lat <= LAT_MAX) and (LON_MIN <= lon <= LON_MAX)
    if not gps_valido:
        motivos.append(f'gps_fuera_chile(lat={lat}, lon={lon})')

    # --- Si hay errores críticos, enviar a inválidos ---
    if motivos:
        yield beam.pvalue.TaggedOutput(self.OUTPUT_INVALIDO, {
            'patente': str(row.get('patente', '')),
            'latitud': lat,
            'longitud': lon,
            'velocidad': vel,
            'servicio': str(row.get('servicio', '')),
            'timestamp_gps': str(row.get('timestamp_gps', '')),
            'archivo_origen': self.archivo_origen,
            'motivo_rechazo': ' | '.join(motivos),
            'procesado_en': ahora_utc.isoformat(),
        })
    return

    # --- Columnas derivadas ---
    santiago = __import__('pytz').timezone('America/Santiago')
    ts_cap_local = ts_cap.astimezone(santiago) if hasattr(ts_cap, 'astimezone')
else ts_cap
    hora = ts_cap_local.hour if ts_cap_local else 0
    dia_en = ts_cap_local.strftime('%A') if ts_cap_local else 'Monday'
    es_hora_punta = (7 <= hora < 9) or (17 <= hora < 19)

```

```

        latencia = None
        if ts_gps and ts_cap:
            latencia = (ts_cap - ts_gps).total_seconds()

        yield {
            'patente':          str(row.get('patente', '')),
            'servicio':         str(row.get('servicio', '')),
            'operador':         int(row.get('operador') or 0),
            'direccion':        str(row.get('direccion', '')),
            'latitud':          lat,
            'longitud':         lon,
            'velocidad':        vel,
            'timestamp_gps':    ts_gps.isoformat() if ts_gps else None,
            'timestamp_captura': ts_cap.isoformat() if ts_cap else None,
            'fecha_particion':  ts_cap_local.strftime('%Y-%m-%d') if ts_cap_local
else None,
            'hora_captura':     hora,
            'dia_semana':       DIAS_ES.get(dia_en, dia_en),
            'es_hora_punta':    es_hora_punta,
            'latencia_segundos': latencia,
            'velocidad_valida': vel_valida,
            'gps_valido':       gps_valido,
            'archivo_origen':   self.archivo_origen,
            'procesado_en':     ahora_utc.isoformat(),
        }

```

```

# -----
# Control de idempotencia
# -----

def ya_fue_procesado(project_id, archivo_nombre):
    """
    Consulta pipeline_log. Retorna True si el archivo ya fue
    procesado exitosamente (estado = 'SUCCESS').
    """
    client = bigquery.Client(project=project_id)
    query = f"""
        SELECT COUNT(*) AS n
        FROM `{project_id}.buses_control.pipeline_log`
        WHERE archivo_nombre = @archivo
        AND estado = 'SUCCESS'
    """
    job_config = bigquery.QueryJobConfig(
        query_parameters=[
            bigquery.ScalarQueryParameter('archivo', 'STRING', archivo_nombre)
        ]
    )
    resultado = client.query(query, job_config=job_config).result()
    return next(iter(resultado)).n > 0

def registrar_log(project_id, archivo, fecha, estado,
                  filas_ok, filas_ko, inicio, fin, error=None):
    """Inserta una fila en pipeline_log con el resultado de la ejecución."""
    client = bigquery.Client(project=project_id)
    tabla = f'{project_id}.buses_control.pipeline_log'
    fila = [{

```

```

        'archivo_nombre':    archivo,
        'fecha_datos':       fecha,
        'estado':            estado,
        'filas_procesadas':   filas_ok,
        'filas_rechazadas':   filas_ko,
        'inicio_proceso':     inicio.isoformat(),
        'fin_proceso':        fin.isoformat(),
        'mensaje_error':      error,
    }]
    errors = client.insert_rows_json(tabla, fila)
    if errors:
        log.warning(f'Error al escribir pipeline_log: {errors}')

```

```

# -----
# Función principal – lee en batches y ejecuta el pipeline Beam
# -----

def ejecutar_pipeline(project_id, bucket, archivo_gcs, region, temp_dir):
    """
    Lee el Parquet en batches de 50.000 filas para no saturar la RAM
    de Cloud Shell. Cada batch se procesa con un pipeline Beam independiente.
    """
    archivo_nombre = os.path.basename(archivo_gcs)
    fecha_str      = archivo_nombre.replace('buses_', '').replace('.parquet', '')

    # --- Control de idempotencia ---
    if ya_fue_procesado(project_id, archivo_nombre):
        log.info(f'[SKIP] {archivo_nombre} ya fue procesado. Omitiendo.')
        return

    inicio = datetime.now(timezone.utc)
    log.info(f'[START] Procesando {archivo_nombre}')

    # --- Abrir Parquet como stream de batches (no carga todo en RAM) ---
    gcs_fs = pafs.GcsFileSystem()
    ruta_gcs = archivo_gcs.replace('gs://', '')
    pf = pq.ParquetFile(ruta_gcs, filesystem=gcs_fs)

    BATCH_SIZE = 50_000    # filas por batch – ajustar si hay OOM

    schema_validos = {
        'fields': [
            {'name': 'patente',          'type': 'STRING'},
            {'name': 'servicio',          'type': 'STRING'},
            {'name': 'operador',          'type': 'INTEGER'},
            {'name': 'direccion',         'type': 'STRING'},
            {'name': 'latitud',           'type': 'FLOAT'},
            {'name': 'longitud',          'type': 'FLOAT'},
            {'name': 'velocidad',         'type': 'FLOAT'},
            {'name': 'timestamp_gps',     'type': 'TIMESTAMP'},
            {'name': 'timestamp_captura', 'type': 'TIMESTAMP'},
            {'name': 'fecha_particion',   'type': 'DATE'},
            {'name': 'hora_captura',       'type': 'INTEGER'},
            {'name': 'dia_semana',        'type': 'STRING'},
            {'name': 'es_hora_punta',     'type': 'BOOLEAN'},
            {'name': 'latencia_segundos', 'type': 'FLOAT'},

```

```

        {'name': 'velocidad_valida', 'type': 'BOOLEAN'},
        {'name': 'gps_valido', 'type': 'BOOLEAN'},
        {'name': 'archivo_origen', 'type': 'STRING'},
        {'name': 'procesado_en', 'type': 'TIMESTAMP'},
    ]
}
schema_invalidos = {
    'fields': [
        {'name': 'patente', 'type': 'STRING'},
        {'name': 'latitud', 'type': 'FLOAT'},
        {'name': 'longitud', 'type': 'FLOAT'},
        {'name': 'velocidad', 'type': 'FLOAT'},
        {'name': 'servicio', 'type': 'STRING'},
        {'name': 'timestamp_gps', 'type': 'STRING'},
        {'name': 'archivo_origen', 'type': 'STRING'},
        {'name': 'motivo_rechazo', 'type': 'STRING'},
        {'name': 'procesado_en', 'type': 'TIMESTAMP'},
    ]
}

opts = PipelineOptions([
    f'--project={project_id}',
    f'--temp_location={temp_dir}',
    '--runner=DirectRunner',
    '--save_main_session',
])

filas_ok = 0
filas_ko = 0
error_msg = None
estado = 'SUCCESS'
batch_num = 0

try:
    for batch in pf.iter_batches(batch_size=BATCH_SIZE):
        batch_num += 1
        registros = batch.to_pylist()
        log.info(f' Batch {batch_num}: {len(registros)} registros')

        with beam.Pipeline(options=opts) as p:
            resultados = (
                p
                | 'Crear' >> beam.Create(registros)
                | 'Transformar' >> beam.ParDo(
                    TransformarRegistro(archivo_nombre)
                ).with_outputs(
                    TransformarRegistro.OUTPUT_INVALIDO,
                    main='validos'
                )
            )
            resultados.validos | 'Escribir validos' >> WriteToBigQuery(
                table=f'{project_id}:buses_dw.bus_positions',
                schema=schema_validos,
                write_disposition=BigQueryDisposition.WRITE_APPEND,
                create_disposition=BigQueryDisposition.CREATE_NEVER,
            )
            resultados.invalido | 'Escribir invalidos' >> WriteToBigQuery(

```

```

        table=f'{project_id}:buses_control.registros_invalidos',
        schema=schema_invalidos,
        write_disposition=BigQueryDisposition.WRITE_APPEND,
        create_disposition=BigQueryDisposition.CREATE_NEVER,
    )

except Exception as e:
    estado = 'FAILED'
    error_msg = str(e)
    log.error(f'[ERROR] {archivo_nombre}: {e}')

fin = datetime.now(timezone.utc)
registrar_log(
    project_id, archivo_nombre, fecha_str,
    estado, filas_ok, filas_ko, inicio, fin, error_msg
)
log.info(f'[DONE] {archivo_nombre} - {batch_num} batches procesados')

# -----
# Entry point
# -----

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='ETL Buses → BigQuery')
    parser.add_argument('--project', required=True, help='GCP Project ID')
    parser.add_argument('--bucket', required=True, help='Nombre del bucket GCS')
    parser.add_argument('--region', default='us-central1')
    parser.add_argument('--fechas', nargs='+',
                        default=['2026-05-08', '2026-05-09', '2026-05-10'],
                        help='Fechas a procesar (YYYY-MM-DD)')
    args = parser.parse_args()

    temp_dir = f'gs://{args.bucket}/temp/dataflow'

    for fecha in args.fechas:
        archivo_gcs =
f'gs://{args.bucket}/raw/buses/date={fecha}/buses_{fecha}.parquet'
        log.info(f'=== Procesando fecha: {fecha} | Archivo: {archivo_gcs} ===')
        ejecutar_pipeline(args.project, args.bucket, archivo_gcs, args.region,
temp_dir)

    log.info('=== Proceso finalizado para todas las fechas ===')
PYEOF

echo 'Archivo pipeline_buses.py creado correctamente.'
wc -l ~/buses_etl/pipeline_buses.py

```

3.3 Ejecutar el Pipeline

El pipeline usa DirectRunner y procesa los datos en batches de 50.000 filas para evitar saturar la RAM de Cloud Shell (límite ~8GB). El archivo mas grande tiene ~63MB y generara ~28 batches. El proceso completo puede tardar 30-60 minutos.

Por que procesamos en batches de 50.000 filas

Cargar 1.37M filas completas en memoria provoca un error Killed del sistema operativo.

Con `iter_batches()` PyArrow lee el Parquet en trozos, nunca superando ~200MB de RAM por batch. Cada batch genera un pipeline Beam independiente que escribe directamente en BigQuery.

Paso 1 — Si ya corriste el pipeline anteriormente, limpia las tablas antes de relanzar. Ejecuta en BigQuery > Editor de consultas:

```
-- Recrear bus_positions sin NOT NULL (requerido por Apache Beam WriteToBigQuery)
DROP TABLE IF EXISTS buses_dw.bus_positions;

CREATE TABLE buses_dw.bus_positions (
  patente          STRING,
  servicio          STRING,
  operador          INT64,
  direccion         STRING,
  latitud           FLOAT64,
  longitud          FLOAT64,
  velocidad         FLOAT64,
  timestamp_gps     TIMESTAMP,
  timestamp_captura TIMESTAMP,
  fecha_particion   DATE,
  hora_captura      INT64,
  dia_semana        STRING,
  es_hora_punta     BOOL,
  latencia_segundos FLOAT64,
  velocidad_valida  BOOL,
  gps_valido        BOOL,
  archivo_origen    STRING,
  procesado_en      TIMESTAMP
)
PARTITION BY fecha_particion
CLUSTER BY servicio, operador, patente;

-- Limpiar tablas de control para reprocesar desde cero
DELETE FROM buses_control.registros_invalidos WHERE TRUE;
DELETE FROM buses_control.pipeline_log WHERE TRUE;
```

Paso 2 — Abre una sesion tmux antes de ejecutar. Cloud Shell corta procesos inactivos despues de 20 minutos(opcional):

```
tmux new-session -s etl
# Si la sesion se desconecta: tmux attach -t etl
```

Paso 3 — Exporta las variables y ejecuta el pipeline dentro de tmux:

```
export PROJECT_ID=$(gcloud config get-value project)
export BUCKET_NAME="buses-datalake-${PROJECT_ID}"
export REGION="us-central1"

cd ~/buses_etl

python pipeline_buses.py \
  --project=$PROJECT_ID \
  --bucket=$BUCKET_NAME \
  --region=$REGION \
  --fechas 2026-05-08 2026-05-09 2026-05-10
```

Deberias ver logs similares a estos mientras corre:

```

INFO: === Procesando fecha: 2026-05-08 ===
INFO: [START] Procesando buses_2026-05-08.parquet
INFO:   Batch 1: 50000 registros
INFO:   Batch 2: 50000 registros
INFO:   Batch 3: 50000 registros
...
INFO: [DONE] buses_2026-05-08.parquet — N batches procesados
INFO: === Procesando fecha: 2026-05-09 ===
...

```

Idempotencia — que pasa si ejecutas el script dos veces

El pipeline consulta pipeline_log al inicio de cada archivo.
Si encuentra estado = SUCCESS, muestra [SKIP] y omite ese archivo.
Puedes relanzar el script con seguridad en cualquier momento.

3.5 Verificar la Carga en BigQuery

Una vez completado el job, verifica que los datos llegaron correctamente:

```

-- Contar registros cargados por fecha de partición
SELECT
  fecha_particion,
  COUNT(*) AS total_registros,
  COUNTIF(velocidad_valida = FALSE) AS invalidos_vel,
  COUNTIF(gps_valido = FALSE) AS invalidos_gps
FROM buses_dw.bus_positions
GROUP BY fecha_particion
ORDER BY fecha_particion;

-- Ver el log de ejecución del pipeline
SELECT
  archivo_nombre, estado,
  filas_procesadas, filas_rechazadas,
  TIMESTAMP_DIFF(fin_proceso, inicio_proceso, SECOND) AS duracion_seg
FROM buses_control.pipeline_log
ORDER BY inicio_proceso DESC;

```

✓ Verificación del Paso 3

El log debe mostrar estado = 'SUCCESS' para los tres archivos.
La tabla bus_positions debe tener registros para las tres fechas de partición.
La tabla registros_invalidos debe tener los registros rechazados con su motivo_rechazo.
Si relanzas el script, los tres archivos deben aparecer como [SKIP] en los logs.

Paso 4 — Análisis SQL y Preguntas de Negocio

Con los datos cargados en BigQuery, crearás vistas SQL que responden a cuatro preguntas de negocio clave sobre la operación de la flota. Estas vistas pueden conectarse a herramientas de visualización como Looker Studio.

💡 ¿Qué es una Vista en BigQuery?

Una vista (VIEW) es una consulta guardada que se ejecuta al ser invocada.

No almacena datos físicamente, pero simplifica el acceso para reportes y dashboards.

Puedes conectar Looker Studio directamente a estas vistas.

Pregunta 1 — ¿Cuál es la velocidad promedio por servicio y hora del día?

Objetivo: Identificar servicios con baja velocidad en horas punta, que podrían indicar congestión recurrente.

```
CREATE OR REPLACE VIEW buses_dw.v_velocidad_por_servicio_hora AS
SELECT
  servicio,
  hora_captura,
  dia_semana,
  es_hora_punta,
  COUNT(*) AS total_registros,
  ROUND(AVG(velocidad), 2) AS velocidad_promedio_kmh,
  ROUND(MIN(velocidad), 2) AS velocidad_minima_kmh,
  ROUND(MAX(velocidad), 2) AS velocidad_maxima_kmh,
  ROUND(STDDEV(velocidad), 2) AS desviacion_std
FROM buses_dw.bus_positions
WHERE velocidad_valida = TRUE
GROUP BY
  servicio, hora_captura, dia_semana, es_hora_punta
ORDER BY
  servicio, hora_captura;
```

```
-- Consulta de ejemplo: TOP 10 servicios más lentos en hora punta
SELECT
  servicio,
  ROUND(AVG(velocidad_promedio_kmh), 1) AS vel_prom_hora_punta
FROM buses_dw.v_velocidad_por_servicio_hora
WHERE es_hora_punta = TRUE
GROUP BY servicio
ORDER BY vel_prom_hora_punta ASC
LIMIT 10;
```

Pregunta 2 — ¿Cuántos buses únicos opera cada empresa por servicio?

Objetivo: Evaluar la distribución de la flota por operador y detectar si algún servicio está sub-cubierto o sobre-dimensionado.

```
CREATE OR REPLACE VIEW buses_dw.v_flota_por_operador AS
SELECT
  operador,
  servicio,
```



```

    fecha_particion,
    COUNT(DISTINCT patente)          AS buses_activos,
    COUNT(*)                          AS total_pings_gps,
    ROUND(COUNT(*) / COUNT(DISTINCT patente), 0) AS pings_por_bus
FROM buses_dw.bus_positions
GROUP BY
    operador, servicio, fecha_particion
ORDER BY
    fecha_particion, operador, buses_activos DESC;

```

Pregunta 3 — ¿Cuál es la latencia entre el GPS del bus y el servidor central?

Objetivo: Detectar buses o servicios con problemas de transmisión de datos, lo que podría indicar fallas en el hardware GPS o en la conectividad.

```

CREATE OR REPLACE VIEW buses_dw.v_latencia_transmision AS
SELECT
    fecha_particion,
    servicio,
    operador,
    COUNT(*)                      AS total_registros,
    ROUND(AVG(latencia_segundos), 1) AS latencia_promedio_seg,
    ROUND(PERCENTILE_CONT(latencia_segundos, 0.5)
          OVER (PARTITION BY fecha_particion, servicio), 1) AS latencia_mediana_seg,
    ROUND(MAX(latencia_segundos), 1) AS latencia_maxima_seg,
    COUNTIF(latencia_segundos > 300) AS registros_con_alta_latencia,
    ROUND(
        COUNTIF(latencia_segundos > 300) * 100.0 / COUNT(*),
        2
    ) AS pct_alta_latencia
FROM buses_dw.bus_positions
GROUP BY
    fecha_particion, servicio, operador
ORDER BY
    latencia_promedio_seg DESC;

```

```

-- Consulta de ejemplo: Servicios con mayor porcentaje de alta latencia
SELECT servicio, operador,
    ROUND(AVG(latencia_promedio_seg), 1) AS lat_prom,
    ROUND(AVG(pct_alta_latencia), 1)    AS pct_problemas
FROM buses_dw.v_latencia_transmision
GROUP BY servicio, operador
HAVING AVG(pct_alta_latencia) > 5
ORDER BY pct_problemas DESC
LIMIT 15;

```

Pregunta 4 — ¿Cómo evoluciona la cobertura geográfica de la flota durante el día?

Objetivo: Entender en qué horarios la flota tiene mayor dispersión geográfica, lo que puede indicar horarios de mayor servicio o zonas con mayor demanda.

```

CREATE OR REPLACE VIEW buses_dw.v_cobertura_geografica AS

```

```

SELECT
  fecha_particion,
  hora_captura,
  es_hora_punta,
  COUNT(DISTINCT patente)      AS buses_activos,
  COUNT(*)                     AS total_pings,

  -- Bounding box geográfico en esa hora
  ROUND(MIN(latitud), 4)       AS lat_min,
  ROUND(MAX(latitud), 4)       AS lat_max,
  ROUND(MIN(longitud), 4)      AS lon_min,
  ROUND(MAX(longitud), 4)      AS lon_max,

  -- Centroides aproximado de la flota
  ROUND(AVG(latitud), 4)       AS lat_centroide,
  ROUND(AVG(longitud), 4)      AS lon_centroide,

  -- Dispersión geográfica (mayor valor = más dispersa)
  ROUND(STDDEV(latitud) + STDDEV(longitud), 5) AS dispersion_geografica

FROM buses_dw.bus_positions
WHERE gps_valido = TRUE
GROUP BY
  fecha_particion, hora_captura, es_hora_punta
ORDER BY
  fecha_particion, hora_captura;

```

```

-- Consulta de ejemplo: Comparar dispersión en hora punta vs. normal
SELECT
  es_hora_punta,
  ROUND(AVG(buses_activos), 0)      AS promedio_buses_activos,
  ROUND(AVG(dispersion_geografica), 5) AS dispersion_promedio
FROM buses_dw.v_cobertura_geografica
GROUP BY es_hora_punta;

```

✓ Verificación del Paso 4

Debes tener 4 vistas creadas en el dataset buses_dw.

Ejecuta cada vista con LIMIT 5 para verificar que retornan datos.

Opcional: Conecta Looker Studio a BigQuery y crea un dashboard con estas vistas.

Verificación y Control de Calidad

Ejecuta estas consultas para validar que tu pipeline procesó los datos correctamente antes de dar por terminado el laboratorio.

Verificación General

```
-- Conteo de registros por fecha de partición
SELECT
  fecha_particion,
  COUNT(*) AS total_registros,
  COUNT(DISTINCT patente) AS buses_unicos,
  COUNT(DISTINCT servicio) AS servicios_unicos,
  COUNTIF(velocidad_valida = FALSE) AS registros_vel_invalida,
  COUNTIF(gps_valido = FALSE) AS registros_gps_invalido
FROM buses_dw.bus_positions
GROUP BY fecha_particion
ORDER BY fecha_particion;
```

Verificar Log de Pipeline

```
-- Ver historial de ejecuciones del pipeline
SELECT
  archivo_nombre,
  fecha_datos,
  estado,
  filas_procesadas,
  filas_rechazadas,
  TIMESTAMP_DIFF(fin_proceso, inicio_proceso, SECOND) AS duracion_segundos
FROM buses_control.pipeline_log
ORDER BY inicio_proceso DESC;
```

Valores Esperados

Total de filas aproximado	~1.377.438 registros entre las 3 fechas
Buses únicos esperados	Entre 500 y 2.000 patentes distintas
Servicios esperados	Entre 50 y 200 códigos de servicio distintos
GPS inválidos esperados	< 1% del total (coordenadas fuera de Chile)
Velocidades inválidas	< 2% del total (velocidad fuera de 0-120 km/h)